

ECE-GY 6143: Introduction to Machine Learning

Midterm Solutions, Fall 2025

Prof. Sundeep Rangan

Name: _____

NetID: _____

Instructions:

- Exam is closed-book, no calculators or electronic aids.
- You are permitted two double-sided cheat sheets.
- Exam time is 150 minutes
- Write answers on the pages. If you run out of pages, please ask the instructor or course assistant for more pages.

Good luck and Happy Thanksgiving!

1. *Linear time-series modeling*: You are given scalar time series data, (x_i, y_i) , $i = 0, 1, \dots, T-1$ and want to fit a model:

$$y_i \approx \frac{ax_i}{1 + b_0x_{i-1} + b_1y_{i-1}}, \quad i = 1, \dots, T-1, \quad (1)$$

with parameters (a, b_0, b_1) . Re-arranging (1), we have:

$$y_i(1 + b_0x_{i-1} + b_1y_{i-1}) \approx ax_i. \quad (2)$$

- (a) Using (2), write a linear model of the form:

$$y_i \approx \hat{y}_i = a\phi_{i1}(x, y) + b_0\phi_{i2}(x, y) + b_1\phi_{i3}(x, y), \quad i = 1, \dots, T-1,$$

for three basis functions $\phi_{i1}(x, y)$, $\phi_{i2}(x, y)$, and $\phi_{i3}(x, y)$. Write $\phi_{i1}(x, y)$ in terms of the values of x_i , x_{i-1} , y_i and y_{i-1} .

- (b) Given python vectors **x** and **y** write python function to create a matrix **A** with columns of $\phi_{i1}, \phi_{i2}, \phi_{i3}$. Hint: Be careful about the indexing of i vs. $i-1$.

You may use that for a vector **z**=(**z**[0],...,**z**[**n**-1]):

```
z[a:] = (z[a], z[a+1], ... , z[n-1])
z[:b] = (z[0], z[1], ... , z[b-1])
```

Also,

```
A = np.column_stack((c0,c1,c2)) # matrix with column c1, c2, c3
```

- (c) Write a python function that, given x and parameters predicts the output $\hat{y}_i$ via the recursion:

$$\hat{y}_i = \frac{ax_i}{1 + b_0x_{i-1} + b_1\hat{y}_{i-1}}, \quad \hat{y}_0 = y_0.$$

You can use the syntax:

```
def predict_rec(x,y0,a,b0,b1):
    ...
    return yhat
```

Inside the function, just loop over **i**.

Solution:

- (a) Re-arranging (2):

$$y_i \approx \hat{y}_i = ax_i - b_0x_{i-1}y_i - b_1y_iy_{i-1},$$

so we can take the basis functions as:

$$\phi_{i1}(x, y) = x_i, \quad \phi_{i2}(x, y) = -x_{i-1}y_i, \quad \phi_{i3}(x, y) = -y_iy_{i-1}.$$

- (b) We can form the matrix as:

```
phi1 = x[1:]
phi2 = -x[:-1]*y[1:]
phi3 = -y[:-1]*y[1:]
A = np.column_stack((phi1,phi2,phi3))
```

(c) We can use a simple loop

```
def predict_rec(x,y0,a,b0,b1):  
    T = len(x)  
    yhat = np.zeros(T)  
    for i in range(1,n):  
        yhat[i] = a/(1 + b0*x[i-1] + b1*yhat[i-1])  
    return yhat
```

2. *Bias*: Suppose we have a model between scalars x and y :

$$\hat{y} = f(x, \theta) = \theta_0 e^{\theta_1 x}$$

for parameters $\theta = (\theta_0, \theta_1)$.

- (a) Given two data points (x_1, y_1) and (x_2, y_2) , find $\hat{\theta}$ such that $y_i = f(x_i, \hat{\theta})$ for $i = 1, 2$.
Hint: Take logarithms and fit a line between the two points.
- (b) Now suppose the two data points came from a true function

$$y_i = f_0(x_i) = 1 + e^{x_i}.$$

Note there is no noise. If the data is $x_1 = 0$ and $x_2 = a$ for some $a > 0$, what are $\hat{\theta}_0$ and $\hat{\theta}_1$ in terms of a ?

- (c) Given any other $x > 0$, what is the bias error:

$$\text{Bias}(x) = f_0(x) - f(x, \hat{\theta}).$$

Write your answer in terms of x and a .

Solution:

- (a) Taking logarithms of both sides of $y_i = f(x_i, \theta)$, we have:

$$y_i = \theta_0 e^{\theta_1 x_i} \Rightarrow \ln(y_i) = \ln(\theta_0) + \theta_1 x_i.$$

This is a line that with slope θ_1 and intercept $\ln(\theta_0)$ that must go through the points $(x_1, \ln(y_1))$ and $(x_2, \ln(y_2))$. Hence:

$$\hat{\theta}_1 = \frac{\ln(y_2) - \ln(y_1)}{x_2 - x_1}, \quad \ln \hat{\theta}_0 = \ln y_1 - \hat{\theta}_1 x_1.$$

The second equation can be re-written as:

$$\begin{aligned} \hat{\theta}_1 &= \frac{\ln(y_2/y_1)}{x_2 - x_1} \\ \hat{\theta}_0 &= y_1 \exp(-\hat{\theta}_1 x_1). \end{aligned}$$

(b) At the data points $x_1 = 0$ and $x_2 = a$, the corresponding values of y_1 and y_2 are:

$$\begin{aligned}x_1 = 0 &\Rightarrow y_1 = e^{x_1} + 1 = e^0 + 1 = 2, \\x_2 = a &\Rightarrow y_2 = e^{x_2} + 1 = e^a + 1.\end{aligned}$$

Using the formulae from part (b):

$$\begin{aligned}\hat{\theta}_1 &= \frac{1}{a} \ln((e^a + 1)/2) \\ \hat{\theta}_0 &= y_1 = 2.\end{aligned}$$

(c) The estimated function is:

$$\begin{aligned}\hat{y} &= f(x, \hat{\theta}) \\ &= \hat{\theta}_0 e^{\hat{\theta}_1 x} = 2 \exp \left[\frac{x}{a} \ln((e^a + 1)/2) \right] \\ &= 2 \left[\frac{e^a + 1}{2} \right]^{x/a}.\end{aligned}$$

The bias error is:

$$\begin{aligned}\text{Bias}(x) &= f_0(x) - f(x, \hat{\theta}) \\ &= e^x + 1 - 2 \left[\frac{e^a + 1}{2} \right]^{x/a}.\end{aligned}$$

It is not required, but we can verify that the bias error at x_1 and x_2 is zero:

$$\begin{aligned}\text{Bias}(0) &= 2 - 2 \left[\frac{e^a + 1}{2} \right]^0 = 2 - 2 = 0 \\ \text{Bias}(a) &= e^a + 1 - 2 \left[\frac{e^a + 1}{2} \right]^1 = e^a + 1 - (e^a + 1) = 0.\end{aligned}$$

3. *Logistic regression:* Consider a logistic model of the form:

$$\begin{aligned}z &= w_1 \phi(x) + w_2 \phi(x - 3) + b \\ \mathbb{P}(y = 1|x) &= \frac{1}{1 + e^z},\end{aligned}$$

where $\phi(x)$ is the triangular function:

$$\phi(x) = \max\{1 - |x|, 0\}.$$

After training, the parameters are found to be $(w_1, w_2) = (1, 2)$ and $b = -0.5$.

- Draw z as a function for x for the parameters above. The function is piecewise linear. Indicate the precise locations of all the line segments.
- Suppose the classifier takes $\hat{y} = 1$ when $\mathbb{P}(y = 1|x) > 0.5$. On the graph in part (a), draw the regions of values x where $\hat{y} = 1$. Determine and indicate all the interval boundaries.

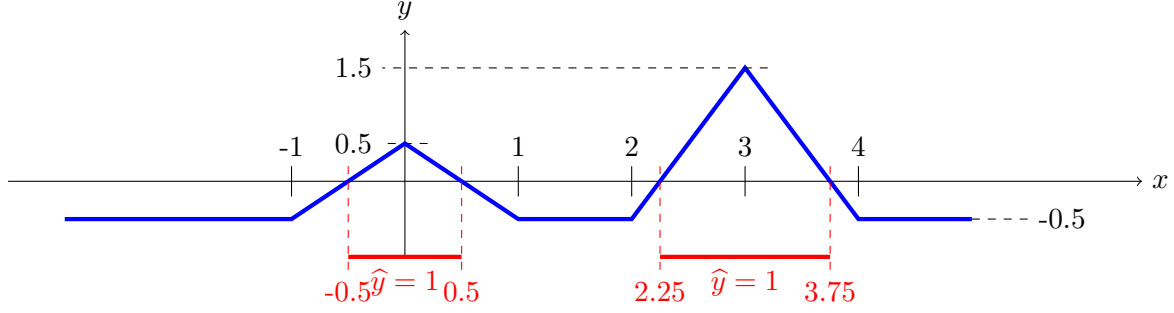


Figure 1: Problem 3 Plot of $z(x) = \phi(x) + 2\phi(x - 3) - 0.5$. Also plotted in red are the decision regions where $\hat{y} = 1$.

- (c) Suppose that for some sample, x is uniformly distributed in the region $x \in [2, 4]$. What $\mathbb{P}(\hat{y} = 1)$?

Solution:

(a) The function z is a sum of two shifted triangles. See Fig. 1.

(b) We have

$$\hat{y} = 1 \iff \mathbb{P}(y = 1|x) > 0.5 \iff z > 0.$$

From Fig. 1, there are two regions where $z > 0$: The first region corresponds to the first triangle. In this region:

$$z = -0.5 + 1 - |x| > 0 \iff |x| < 0.5 \iff x \in (-0.5, 0.5).$$

The second region corresponds to the second triangle. In this region:

$$z = -0.5 + 2(1 - |x - 3|) > 0 \iff 2|x - 3| < 1.5 \iff x \in (2.25, 3.75).$$

So, in total, we have:

$$\hat{y} = 1 \iff x \in (-0.5, 0.5) \cup (2.25, 3.75).$$

The two regions are indicated in red in Fig. 1.

- (c) In this region, $\hat{y} = 1$ when $x \in (2.25, 3.75)$. Since the x is uniformly distributed, the probability is proportional to the area:

$$\mathbb{P}(\hat{y} = 1) = \mathbb{P}(x \in (2.25, 3.75)) = \frac{3.75 - 2.25}{4 - 2} = \frac{1.5}{2} = 0.75.$$

4. *Gradients*: We wish to fit an inverse model:

$$\hat{y}_i = \frac{1}{1 + \sum w_j x_{ij}}, \quad (3)$$

with a mean squared error metric:

$$J(\mathbf{w}) = \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

- (a) Use chain rule to find the partial derivatives $\partial J / \partial w_j$.
- (b) Complete the function in python that computes the function and its gradient:

```
def Jeval(...):
    ....
    return J, Jgrad
```

You need to specify the arguments for the `Jeval`.

Solution:

- (a) Let $z_i = 1 + \sum w_j x_{ij}$ so that $\hat{y}_i = 1/z_i$. Then:

$$\begin{aligned} \frac{\partial J}{\partial w_j} &= \sum_i \frac{\partial J}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial z_i} \frac{\partial z_i}{\partial w_j} \\ &= - \sum_i 2(y_i - \hat{y}_i)(-1) \frac{1}{z_i^2} x_{ij} = \sum_i (y_i - \hat{y}_i) \frac{1}{z_i^2} x_{ij} \end{aligned}$$

- (b) The most efficient implementation of the function is:

```
def Jeval(X,w):
    z = 1+X.dot(w)
    yhat = 1/z
    J = 2*np.sum((y-yhat)**2)

    dJ_dz = (y-yhat)/(z**2)
    Jgrad = dJ_dz[:,None]*X

    return J, Jgrad
```

But, you will get full marks if you use loops. For example:

```
def Jeval(X,w):
    z = X.dot(w)
    yhat = 1/z
    J = np.sum((y-yhat)**2)

    dJ_dz = 2*(y-yhat)/(z**2)

    n, d = X.shape
    Jgrad = np.zeros((n,d))
    for i in range(n):
        Jgrad[i,:] = dJ_dz[i]*X[i,:]
```

```
return J, Jgrad
```

5. *Nearest neighbor as a pre-step*: You are given two sets of data samples $\mathbf{x}_i \in \mathbb{R}^p$, $i = 1, \dots, n$ and $\mathbf{q}_j \in \mathbb{R}^p$, $j = 1, \dots, m$. They are given as python matrices $\mathbf{X}$ and $\mathbf{Q}$.

- (a) Write python code to create a matrix $\mathbf{Qhat}$ with rows:

$$\hat{\mathbf{q}}_i = \arg \min_{\mathbf{q} \in \{\mathbf{q}_1, \dots, \mathbf{q}_m\}} \|\mathbf{q} - \mathbf{x}_i\|,$$

and a vector $\mathbf{dhat}$ with values:

$$\hat{d}_i = \|\hat{\mathbf{q}}_i - \mathbf{x}_i\|^2.$$

That is, for each i , $\hat{\mathbf{q}}_i$ is the nearest neighbor in the set $\mathbf{q} \in \mathbf{q}_1, \dots, \mathbf{q}_m$ to $\mathbf{x}_i$ and d_i is the minimum distance. You can use the function:

```
np.argmin(d) % returns i where d[i] is minimum
```

- (b) Now suppose we want to fit a model of the form:

$$\hat{y}_i = \mathbf{w}^\top \hat{\mathbf{q}}_i + \alpha \hat{d}_i,$$

for parameters $\mathbf{w} \in \mathbb{R}^p$ and scalar α . So, the prediction is a linear function of the nearest neighbor $\hat{\mathbf{q}}_i$ and distance $\hat{d}_i$. Using the arrays $\mathbf{Qhat}$ and $\mathbf{d}$ and target vector $\mathbf{y}$, write python code to find parameters $\mathbf{w}$ and α . You are given functions:

```
beta = np.linalg.lstsq(A,y) # Finds the least squares solution to ||y-A.dot(beta)||
A = np.hstack((A1,A2)) # Stacks matrices A1 and A2 side-by-side
```

Solution:

- (a) One potential solution is:

```
n, p = X.shape
m = Q.shape[0]
d = np.zeros(m)
Qhat = np.zeros((n,p))
dhat = np.zeros(n)
for i in range(n):
    for j in range(m):
        d[j] = np.sum((X[i,:] - Q[j,:])**2)
    imin = np.argmin(d)
    Qhat[i,:] = Q[imin,:]
    dhat[i] = d[imin]
```

- (b) The most direct way to do the fit is:

```
A = np.hstack((Qhat, dhat))
beta = np.linalg.lstsq(A, y)
w = beta[:p]
```

```
alpha = beta[p]
```